A Project report on

Distributed Approach for efficiently extracting common patterns from large datasets

A Dissertation submitted to JNTU Hyderabad in partial fulfillment of the academic requirements for the award of the degree.

Bachelor of Technology

in

Computer Science and Engineering

Submitted by

CH. ASHWITH
(20H51A0586)

G. SONIYA
(20H51A05C2)

N.PRANAY
(20H51A05A2)

Under the esteemed guidance of

Dr. P. SENTHIL

Assistant Professor

Department of Computer Science and Engineering

CMR COLLEGE OF ENGINEERING & TECHNOLOGY

(UGC Autonomous)

*Approved by AICTE *Affiliated to JNTUH *NAAC Accredited with A^+ Grade

KANDLAKOYA, MEDCHAL ROAD, HYDERABAD - 501401.

2020- 2024

CMR COLLEGE OF ENGINEERING & TECHNOLOGY

KANDLAKOYA, MEDCHAL ROAD, HYDERABAD – 501401

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the Major Project Phase I report entitled **" A Distributed Approach for efficiently extracting common patterns from large datasets"** being submitted by Ch. Ashwith (20H51A0586), G. Soniya (20H51A05C2), N. Pranay (20H51A05A2) in partial fulfillment for the award of **Bachelor of Technology in Computer Science and Engineering** is a record of bonafide work carried out his/her under my guidance and supervision.

The results embodies in this project report have not been submitted to any other University or Institute for the award of any Degree.

Dr. P. Senthil
Assistant Professor
Dept. of CSE

Dr. Siva Skandha Sanagala
Associate Professor and HOD
Dept. of CSE

ACKNOWLEDGEMENT

With great pleasure we want to take this opportunity to express my heartfelt gratitude to all the people who helped in making this project work a grand success.

We are grateful to **Dr. P. Senthil**, **Assistant Professor** Department of Computer Science and Engineering for his valuable technical suggestions and guidance during the execution of this project work.

We would like to thank **Dr. Siva Skandha Sanagala,** Head of the Department of Computer Science and Engineering, CMR College of Engineering and Technology, who is the major driving forces to complete my project work successfully.

We are very grateful to **Dr. Vijaya Kumar Koppula**, Dean-Academics, CMR College of Engineering and Technology, for his constant support and motivation in carrying out the project work successfully.

We are highly indebted to **Major Dr. V A Narayana,** Principal, CMR College of Engineering and Technology, for giving permission to carry out this project in a successful and fruitful way.

We would like to thank the **Teaching & Non- teaching** staff of Department of Computer Science and Engineering for their co-operation

We express our sincere thanks to **Shri. Ch. Gopal Reddy**, Secretary, CMR Group of Institutions, for his continuous care.

Finally, We extend thanks to our parents who stood behind us at different stages of this Project. We sincerely acknowledge and thank all those who gave support directly and indirectly in completion of this project work.

Ch. Ashwith	20H51A0586
G. Soniya	20H51A05C2
N. Pranay	20H51A05A2

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	LIST OF FIGURES	ii
	ABSTRACT	iii
1	**INTRODUCTION**	**1**
	1.1 Problem Statement	2
	1.2 Research Objective	2
	1.3 Project Scope and Limitations	3
2	**BACKGROUND WORK**	**4**
	2.1. FP Mining On Big Data Using MapReduce	5
	2.1.1.Introduction	5
	2.1.2.Merits,Demerits and Challenges	5
	2.1.3.Implementation	7
	2.2. Association Rule Mining	8
	2.2.1.Introduction	8
	2.2.2.Merits,Demerits and Challenges	8
	2.2.3.Implementation	10
	2.3. Parallel Frequent Pattern Mining	11
	2.3.1.Introduction	11
	2.3.2.Merits,Demerits and Challenges	11
	2.3.3.Implementation	12
3	**RESULTS AND DISCUSSION**	**13**
	4.1. Performance metrics	14
4	**CONCLUSION**	**15**
	5.1 Conclusion	16
	REFERENCES	**17**

List of Figures

FIGURE NO.	TITLE	PAGE NO.
2.1	The Association Rules Data Mining Architecture	9
3.1	Standard and Parallel & Distributed Data Mining	1

ABSTRACT

In recent times, knowledge discovery in databases has become a valuable tool for extracting meaningful and useful information. Frequent pattern mining and association rule mining have been widely explored for various real-life applications. However, traditional mining algorithms that rely on centralized and memory-resident data face limitations in terms of handling large datasets, bandwidth constraints, and energy consumption in distributed databases, especially in the context of big data.

To address these challenges and enhance performance, we present a set of algorithms based on FP growth. These algorithms efficiently discover frequent patterns (FPs) while offering fast and scalable service in distributed computing environments. Additionally, we propose a compact data structure for storing items and counts, reducing data transmission overhead on the network.

CHAPTER 1
INTRODUCTION

CHAPTER 1

INTRODUCTION

1.1 Problem Statement

The FP-growth algorithm is not efficient when processing a big database. Hence, numerous algorithms related to distributed or parallel computing have emerged. However, these methods have high communication cost, insufficient memory, low computing efficiency, and redundant computing waste. Therefore, DFP was proposed based on FP growth. If the first stage failed during mining, then the proposed method would estimate the necessary memory and nodes for further distributed computing steps. The excellent performance of DFP is presented in the Experiment Results section. If the memory of the nodes is less than the estimated necessary memory, then the extended database projection method called mixing projection (MP) will be activated. MP is used to store compressed crucial information and develop an efficient FP tree-based mining method. MP performs better than the DP algorithm especially when the process is unable to mine any further.

1.2 Research Objective

Two well-known algorithms, namely, Apriori and FP growth , are proposed to mine frequent itemsets and association rules on the basis of generation and test or pattern growth approaches. The Apriori approach generates a great number of candidate datasets and repetitively scans a database to verify whether a pattern appears frequently or not. Hen et al. proposed the FP-growth method based on FP tree for mining FPs. FP growth scans the databases only twice and the mined information could be obtained from the proposed data structure. However, finding FPs increases the execution time when the database size is large.

1.3 Project Scope and Limitations

Scope:

1. **Scalability:** Distributed approaches are well-suited for handling massive datasets that cannot be processed on a single machine. They can efficiently scale out to accommodate increasing data sizes.
2. **Performance:** These approaches can significantly improve the speed of pattern extraction by parallelizing the task across multiple nodes or machines.
3. **Large Dataset Analysis**: Distributed pattern mining is essential for applications involving vast datasets, such as social media analytics, genomic data analysis, financial transaction data, and more.
4. **Identifying Common Patterns**: This approach can efficiently extract common patterns, trends, or associations in data, enabling insights and decision-making.
5. **Fault Tolerance:** Distributed systems can be designed to be fault-tolerant, ensuring that if one machine fails, the process can continue without data loss.

Limitations:

1. **Implementation Complexity**: Building and managing a distributed system is complex and requires expertise in distributed computing, data engineering, and distributed data storage systems.
2. **Cost**: The hardware and infrastructure required for distributed computing can be expensive, including the cost of setting up clusters of machines and maintaining them.
3. **Data Privacy and Security:** Distributed systems must address data privacy and security concerns, particularly when dealing with sensitive or regulated data.
4. **Data Distribution**: Distributing data across multiple nodes can lead to data skew issues, where some nodes may have more data than others, affecting the efficiency of pattern extraction.

CHAPTER 2
BACKGROUND WORK

CHAPTER 2

BACKGROUND WORK

2.1 Frequent Pattern Mining on Big Data Using MapReduce Programming Model

2.1.1 Introduction

Frequent pattern mining is a fundamental task in data mining and has numerous applications in various fields, such as market basket analysis, recommendation systems, and bioinformatics. With the proliferation of big data, traditional data mining techniques face challenges in terms of scalability and efficiency. To address these issues, MapReduce, a parallel and distributed programming model, has gained popularity for processing large-scale data. In this context, this project explores the use of the MapReduce programming model for frequent pattern mining on big data. This introduction section provides an overview of the topic, followed by discussions on the merits, demerits, and challenges associated with this approach.

2.1.2 Merits, Demerits and Challenges

Merits :

1. **Scalability**: MapReduce allows for the distributed processing of large datasets across clusters of computers, enabling efficient analysis of big data.
2. **Parallel Processing**: By breaking down the mining process into map and reduce tasks, it's possible to exploit parallelism, which speeds up the frequent pattern mining process.
3. **Fault Tolerance**: MapReduce provides built-in fault tolerance, ensuring the processing of large datasets can continue even in the presence of node failures.
4. **Cost-Effective**: MapReduce can be implemented on commodity hardware, making it a cost-effective solution for big data processing.
5. **Flexibility**: The MapReduce model is versatile and can be applied to various types of frequent pattern mining algorithms, making it adaptable to different data mining tasks.

Demerits:

1. **Complexity**: Implementing frequent pattern mining algorithms in MapReduce can be complex, requiring a deep understanding of both the mining algorithms and the MapReduce framework.
2. **Data Shuffling Overhead**: The MapReduce model involves a significant amount of data shuffling between mappers and reducers, which can lead to high network overhead.
3. **I/O Bottleneck**: Large-scale data mining often involves reading and writing substantial amounts of data to and from the distributed file system, potentially causing I/O bottlenecks.
4. **Limited Real-Time Processing:** MapReduce is not suitable for real-time data mining as it processes data in batches, which may not be suitable for applications requiring immediate insights.

Challenges:

1) **Algorithm Adaptation:** Adapting traditional frequent pattern mining algorithms to the MapReduce model can be challenging and may require significant algorithmic redesign.
2) **Data Skew:** Data skew, where some keys have significantly more data to process than others, can lead to performance issues and may require advanced partitioning and load balancing techniques.
3) **Resource Management:** Efficiently managing the resources in a MapReduce cluster, such as memory and computing power, is crucial to achieving optimal performance.
4) **Scalability Issues:** While MapReduce is designed for scalability, there can be limitations when dealing with extremely large datasets and complex algorithms, which may necessitate further optimizations.
5) **Security and Privacy:** Ensuring data security and privacy in a distributed environment, especially in scenarios involving sensitive data, is a persistent challenge.

2.1.3 Implementation

The implementation of frequent pattern mining on big data using the MapReduce programming model involves the following steps:

1. **Algorithm Selection**: Choose a suitable frequent pattern mining algorithm, such as Apriori, FP-growth, or Eclat, and adapt it to the MapReduce framework.
2. **Data Preprocessing**: Prepare the big data for processing, including data cleaning, transformation, and formatting to fit the MapReduce input format.
3. **Map and Reduce Tasks**: Implement the frequent pattern mining algorithm by defining the map and reduce tasks. The map tasks generate intermediate results, and the reduce tasks consolidate these results to find frequent patterns.
4. **Data Partitioning**: Divide the input data into splits and distribute them across the MapReduce cluster.
5. **Resource Configuration**: Configure the cluster's resources to ensure efficient processing, considering factors like memory, CPU, and network bandwidth.
6. **Optimization**: Apply optimization techniques to handle data skew, minimize I/O, and improve overall performance.
7. **Testing and Evaluation**: Thoroughly test the implementation on large datasets to ensure correctness and efficiency. Evaluate the results based on performance metrics and the quality of mined patterns.

In conclusion, leveraging the MapReduce programming model for frequent pattern mining on big data offers numerous advantages in terms of scalability and parallel processing. However, it also poses challenges related to algorithm adaptation, data skew, and resource management. The successful implementation of this approach requires a deep understanding of both data mining and distributed computing concepts.

2.2 Association Rule Mining

2.2.1 Introduction

Association rule mining is a fundamental technique in data mining and is often used in various applications such as market basket analysis, recommendation systems, and fraud detection. It helps identify associations or dependencies between items in a dataset. The primary goal is to find frequent itemsets, which are sets of items that appear together in transactions more often than expected by chance, and generate association rules that describe these associations.

2.2.2 Merits, Demerits, Challenges of Association Rule Mining:

Market Basket Analysis: Association rule mining is widely used in retail for market basket analysis. Retailers can identify which products are frequently purchased together, enabling them to optimize store layouts, cross-sell products, and run targeted promotions.

Cross-Selling and Recommendations: E-commerce platforms and recommendation systems use association rules to suggest related products to customers. If a customer buys a camera, for example, the system might recommend camera accessories.

Medical Diagnosis: In healthcare, association rule mining can be used to discover relationships between symptoms and diseases. Identifying co-occurring symptoms can aid in diagnosing medical conditions.

Anomaly Detection: It can be used to detect anomalies or irregular patterns in data, such as identifying potentially fraudulent transactions by recognizing unusual associations.

Demerits of Association Rule Mining:

Curse of Dimensionality: Association rule mining can be computationally expensive in high-dimensional data, making it impractical for datasets with many attributes.

Scalability: It may not scale well to very large datasets. Mining associations in large datasets can be time-consuming and resource-intensive.

Quality of Rules: Mining all possible rules may result in many trivial or uninteresting rules. Filtering and selecting meaningful rules can be challenging.

Privacy Concerns: In some cases, mining associations can reveal sensitive information. Protecting individual privacy while extracting useful knowledge is a significant concern.

Challenges:

Apriori Algorithm: Many association rule mining algorithms, like Apriori, have been developed. Selecting the most appropriate algorithm for a given dataset can be a challenge.

Threshold Setting: Determining the support and confidence thresholds for meaningful rules is subjective and requires domain knowledge.

Handling Categorical and Numerical Data: Handling mixed data types and continuous numerical attributes in association rule mining is complex.

Big Data: Mining associations in big data environments with distributed systems and parallel processing presents challenges in terms of efficiency and scalability.

2.2.3 Implementation of Association Rule Mining:

To implement association rule mining, you typically follow these steps:

Data Preparation: Preprocess and clean your dataset, converting it into a format suitable for association rule mining.

Frequent Itemset Generation: Use an algorithm like Apriori to discover frequent itemsets in your data.

Rule Generation: Based on frequent itemsets, generate association rules, including measures like support and confidence.

Rule Evaluation and Filtering: Evaluate the generated rules and filter out those that are meaningful for your application. Adjust thresholds if necessary.

Application: Apply the rules to make business decisions or recommendations.

Monitoring and Maintenance: Periodically re-run the association rule mining process to adapt to changing data patterns.

Different software libraries and tools, such as Python's mlxtend or the Weka software, can be used to implement association rule mining in practice.

Remember that the success of association rule mining depends on the quality of your data, domain expertise, and the selection of appropriate parameters for the algorithm.

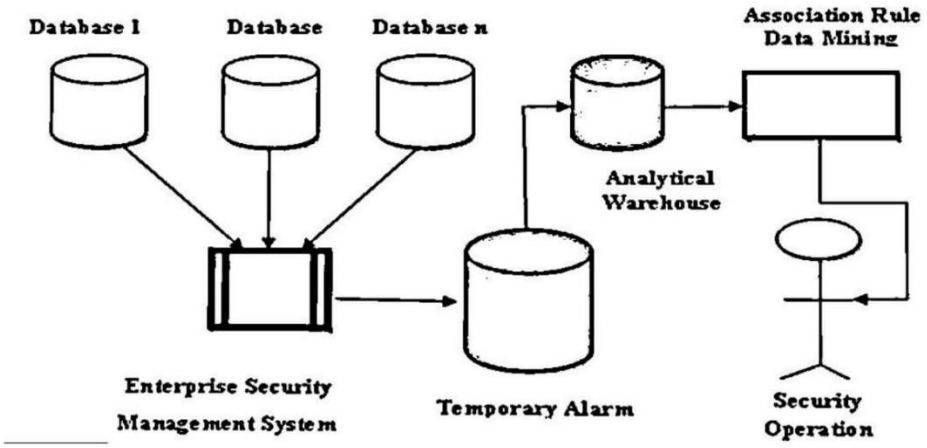


Fig 2.1. The Association Rules Data Mining Architecture

2.3 Parallel Frequent Pattern Mining (PFP):

2.3.1 Introduction

Parallel Frequent Pattern mining is an approach to efficiently identify frequent itemsets in large datasets by leveraging parallel and distributed computing techniques. Frequent itemsets are sets of items that appear together in transactions above a specified support threshold. PFP mining allows for the identification of these patterns in a distributed and scalable manner.

2.3.2 Merits, Demerits and Challenges

Merits:

1. **Scalability:** PFP mining can handle large datasets that might be impractical for single-machine data mining algorithms. It takes advantage of parallel and distributed computing resources for efficient processing.
2. **Speed:** By utilizing multiple processors or nodes, PFP mining can significantly speed up the process of finding frequent patterns, which is especially crucial for real-time or near-real-time applications.
3. **High Throughput:** Parallelization enables a higher throughput of data processing, making it feasible to mine frequent patterns in real-world big data scenarios.

Demerits:

1) **Complex Implementation:** Implementing PFP mining often requires expertise in parallel and distributed computing, which may be a barrier for some users.
2) **Resource Intensive:** It relies on distributed infrastructure, which can be costly in terms of hardware and maintenance.
3) **Load Balancing:** Ensuring even distribution of tasks among parallel resources and load balancing can be challenging in a distributed environment.

Challenges:

1) **Scalability:** While PFP mining is designed for scalability, managing a large number of nodes or processors efficiently is a significant challenge.
2) **Data Distribution:** Ensuring that data is distributed optimally across nodes to minimize data transfer and communication overhead is a key challenge.
3) **Fault Tolerance:** Handling failures in a distributed environment and ensuring the continuity of mining tasks is a critical challenge.

2.3.3 Implementation of PFP Mining:

The implementation of Parallel Frequent Pattern (PFP) mining involves setting up a distributed environment and employing parallel computing frameworks. Common steps include:

1. **Data Distribution:** Distributing the dataset across multiple nodes or processors.
2. **Parallel Mining Algorithms:** Utilizing parallel and distributed algorithms designed for frequent pattern mining.
3. **Load Balancing:** Balancing the workload across distributed resources.
4. **Result Aggregation:** Aggregating results from multiple nodes to obtain the final frequent patterns.
5. **Scalable Infrastructure:** Deploying a scalable infrastructure with the necessary hardware and software components.

   Tools and libraries designed for distributed computing, such as Apache Hadoop and Spark, are often used in the implementation of PFP mining.

   Please note that the specific details and techniques for PFP mining can vary based on the and technologies used, as well as the characteristics of the dataset and the desired outcomes.

CHAPTER 3
RESULTS AND DISCUSSION

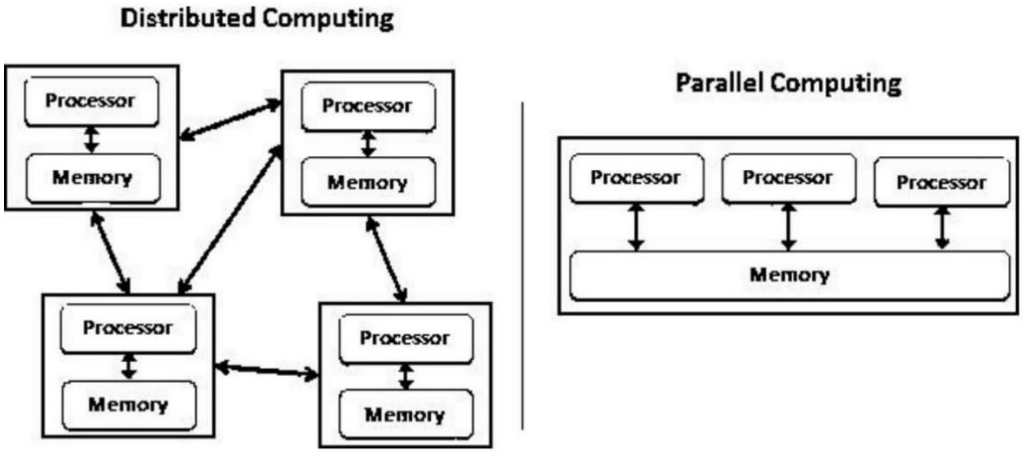
CHAPTER 3

RESULTS AND DISCUSSION

The results of these existing solutions highlight the significance of distributed methods for frequent pattern mining from big data. They offer scalability, efficiency, and applicability in various domains. However, challenges exist in terms of implementation complexity, resource utilization, and privacy concerns.

When choosing an approach, it's essential to consider the specific requirements of the task at hand. For instance, MapReduce-based frequent pattern mining is well-suited for scalable batch processing, while parallel frequent patterns offer real-time capabilities. Association rule mining, on the other hand, is ideal for retail and market basket analysis but must address privacy issues.

Ultimately, the choice of method depends on the characteristics of the dataset, the desired outcomes, and the resources available for implementation. Further research and advancements in these methods are expected to address existing challenges and broaden their applicability in the era of big data



.

Fig 3.1. Standard and Parallel & Distributed Data Mining.

CHAPTER 4
CONCLUSION

CHAPTER 4

CONCLUSION

The exponential growth of big data has necessitated the development of distributed methods for fast mining of frequent patterns. Existing solutions have demonstrated significant progress in addressing the challenges posed by large and complex datasets. These solutions leverage distributed computing frameworks, data partitioning techniques, pruning strategies, and data compression methods to enhance the efficiency and scalability of frequent pattern mining.

In conclusion, while existing solutions have made substantial strides in addressing the challenges of big data mining, there are still areas for improvement. As data continues to grow, it is imperative to continue researching and developing more efficient and scalable algorithms and approaches. Additionally, exploring how emerging technologies like distributed machine learning and cloud computing can further enhance the speed and effectiveness of frequent pattern mining will be essential for staying at the forefront of this field.

The choice of the most appropriate solution depends on the specific characteristics of the data, the available computing resources, and the application requirements. In practice, selecting the right method and fine-tuning it for a particular use case is crucial to effectively mine frequent patterns from big data.

.

REFERENCES

REFERENCES

[1] A Distributed Method for Fast Mining Frequent Patterns From Big data peng-yu huang 1, wan-shu cheng2, ju-chin chen1, wen-yu chung1, young-lin chen3, kawuu w. lin Received September 3, 2021, accepted September 17, 2021, date of publication September 24, 2021, date of current version October 8, 2021.Digital Object Identifier 10.1109/ACCESS.2021.3115514

[2] J. Han, J. Pei, Y. Yin, and R. Mao, ‘‘Mining frequent patterns without candidate generation: A freq+uent-pattern tree approach,’’ Data Mining Knowl. Discovery, vol. 8, no. 1, pp. 53–87, 2004.

[3] R. Agrawal, T. Imieliński, and A. Swami, ‘‘Mining association rules between sets of items in large databases,’’ in Proc. ACM SIGMOD Int. Conf. Manage. Data (SIGMOD), 1993, pp. 207–216.

[4] J. Han, J. Pei, and Y. Yin, ‘‘Mining frequent patterns without candidate generation,’’ ACM SIGMOD Rec., vol. 29, no. 2, pp. 1–12, 2000.

[5] A. Tehreem, S. G. Khawaja, M. U. Akram, S. A. Khan, and M. Ali, ‘‘Parallel architecture for implementation of frequent itemset mining usingFPgrowth,’’

[6] H. Li, Y. Wang, D. Zhang, M. Zhang, and E. Y. Chang, ‘‘Pfp: Parallel fpgrowth for query recommendation,’’ in Proc. ACM Conf. Recommender Syst. (RecSys), 2008, pp. 107–114.

[7] X. Wei, Y. Ma, F. Zhang, M. Liu, and W. Shen, ‘‘Incremental Fpgrowthmining strategy fordynamic ThresholdvalueandatabasedonMapReduce,’’inProc.IEEE18thInt.Conf.Comput.SupportedCooperat. Work Design (CSCWD), May 2014, pp. 271–276.

GitHub Link

1.